# Legal-Hold & Custodian Management Program — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

# Contents


Executive Summary — Legal-Hold & Custodian Management Program (Microsoft Purview) 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 5

Boundaries (so scope is clear) . . . 5

1. Solution Overview — Legal-Hold & Custodian Management Program (Microsoft Purview) 5

What this solution is . . . 5

What this is NOT — the deferred boundary and the MSEDISC boundary . . . 6

The core distinction — hold types in Microsoft Purview . . . 6

Why this matters . . . 7

How it works (the program) . . . 7

The reference design . . . 8

What you can control . . . 9

Limitations (be honest with buyers) . . . 9

Naming convention (used in every doc and script) . . . 10

The standard deployment model . . . 11

2. Requirements — Legal-Hold & Custodian Management Program (Microsoft Purview) 11

2.1 Licensing sufficient for the eDiscovery tier you will use — Standard vs Premium . . . 11

2.2 eDiscovery least-privilege role-group membership . . . 12

2.3 In-scope custodians, data sources, and workloads identified (by legal, implemented here) . . . 12

2.4 Preservation scope and hold-notice content defined by legal (drafted by legal, configured here) 13

2.5 A named release authority and change-control path (holds are preservation-critical) . . . 13

2.6 Audit logging enabled and a pilot matter defined . . . 14

Global Requirements — Microsoft Purview Solutions 15

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 15

2. Purview roles (least-privilege) and portal access . . . 15

3. Security & Compliance PowerShell + Graph toolchain . . . 16

4. Data sources & scope in scope . . . 16

5. Compliance, records, privacy & legal stakeholders . . . 16

6. A pilot scope and change control . . . 17

3. Implementation Guide — Legal-Hold & Custodian Management Program (Microsoft Purview)

Phase 0 — Connect and baseline (Assess) . . . 18

Phase 1 — Design the matter case and preservation scope (Design) . . . 18

Phase 2 — Configure the case hold in the pilot (Configure — preview first) . . . 19

Phase 3 — Add custodians and issue the hold notice (Configure — Premium) . . . 19
Phase 4 — Validate the hold and the notice (Validate) . . . 20
Phase 5 — Track, then expand to live matters (Operate → Expand) . . . 21
Order-of-operations summary . . . 21
5. Testing & Validation — Legal-Hold & Custodian Management Program (Microsoft Purview) 22
Test A — Preservation is effective (the hold actually holds) . . . 22
Test B — Custodians & notices are covered and tracked (Premium) . . . 23
Test C — Operable & defensible (the control can be run and proven) . . . 24
Simulation-first & safety validation . . . 25
Evidence to capture (for the deliverable record) . . . 26
Troubleshooting — Legal-Hold & Custodian Management Program 26
The hold saves but content isn't preserved . . . 26
"Distribution pending / partial" never completes . . . 27
Can't add custodians / custodian communications missing . . . 27
"Access denied" creating or viewing a case/hold . . . 27
Overlapping holds — worried about releasing preserved data . . . 27
Litigation Hold vs case hold confusion . . . 28
Pilot cleanup left a hold in place . . . 28
Government / sovereign cloud . . . 28
7. Best Practices — Legal-Hold & Custodian Management Program (Microsoft Purview) 28
Prove it on a pilot matter before a real one depends on it . . . 28
Preserve first, scope later — default to everything . . . 29
Validate distribution — a hold placed is not a hold effective . . . 29
Notice, acknowledge, remind, escalate — the evidence is the point . . . 29
Release is the one irreversible act — gate it, never script it . . . 29
Respect the legal-vs-technical boundary . . . 29
Licensing & portal reality . . . 30
Operations & governance . . . 30
8. Deliverables Summary — Legal-Hold & Custodian Management Program (Microsoft Purview) 30
Documentation . . . 31
PowerShell tools (5) — Security & Compliance PowerShell / Graph . . . 31
Definition of done . . . 33
Appendix A — PowerShell Scripts 34

# Executive Summary — Legal-Hold & Custodian Management Program (Microsoft Purview)

## The problem

When litigation, an investigation, or a regulatory inquiry arises, the organization has a legal **duty to preserve** the potentially relevant content of the people involved — and to be able to **prove** it did. In most organizations that duty is met **ad hoc**: a preservation email sent to a handful of people, no enforcement on the platform, no record of who acknowledged, no way to show a court the hold was issued, tracked, and released deliberately. When a custodian's mailbox is purged, a leaver's data is deleted, or a hold is simply missed, the content is gone — and **spoliation** sanctions land on the organization. A preservation duty that lives in an email is not preservation.

## The solution

Stand up a repeatable **legal-hold and custodian-management program** in **Microsoft Purview eDiscovery**, using only the Purview portal, Security & Compliance PowerShell, and Microsoft Graph. Each matter gets an **eDiscovery case**; the people of interest are added as **custodians** with their data sources mapped (mailbox, OneDrive, associated Teams/SharePoint); their content is placed on a **case hold** so nothing can be permanently purged; each custodian is issued a **legal-hold notice** they must **acknowledge**, with **reminders and escalation** for non-responders; and the whole thing is **tracked** in a defensible **hold register**. The defining discipline is **pilot-first**: the mechanics — hold distribution, notice issuance, acknowledgment tracking — are proven on a **pilot matter with fabricated test custodians** before any real matter's preservation depends on them. The result: preservation is done consistently, evidenced, and defensible instead of improvised.

## What it delivers

- A **matter case + case hold** (the preservation floor that works at every tier), with the choice of **preserve-everything** or a legally-authorized **query-based** scope.
- **Custodian management** (Premium) — custodians with mapped data sources, plus explicitly-held **non-custodial** locations — so nothing responsive is left un-preserved.
- A **hold-notice workflow** (Premium) — issuance, acknowledgment tracking, reminder, and escalation — the proof a court expects that the hold reached the custodian.
- A **defensible hold register** and an evidence trail: who is on hold, since when, noticed, acknowledged, released.
- **Five PowerShell tools** (posture discovery, hold/custodian config helper, validation checker, tracking report, scope helper) — read-only or `-WhatIf` by default, **no tool releases a hold**, no credentials in code.

## Business outcomes

• **Preservation that holds** — held content cannot be permanently purged, enforced by the platform with an

audit trail, across mailbox, OneDrive, Teams, and SharePoint.

• **Defensibility by evidence** — issuance, acknowledgment, reminder, escalation, and release are tracked, so

the organization can *prove* it met its duty, not just assert it.

• **Repeatable and scalable** — a named, standardized case/hold/notice program with a tracked register runs

the same way every matter, instead of ad-hoc holds that don't scale across a portfolio.

• **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented and

tooled to run in-house.

### Boundaries (so scope is clear)

Custodian management and the hold-notice workflow are **eDiscovery (Premium)** — generally **Microsoft 365 E5**, the **E5 Compliance** add-on, or **E5 eDiscovery & Audit**; **case holds** are available at **E3** without custodians or communications (**verify your tenant**). This is **not** the eDiscovery *readiness assessment*, and collection/review/export are downstream phases — this program stops at **defensible preservation**. Crucially, **legal owns the determination**: *whether* a duty has attached, *who* is a custodian, the *scope* of preservation, a notice's legal sufficiency, and *when* a hold may be released are **legal / eDiscovery-counsel** decisions. This solution provides the **technical program and the evidence**, not the legal judgement — and **it never releases a hold automatically**.

## 1. Solution Overview — Legal-Hold & Custodian Management Program (Microsoft Purview)

### What this solution is

When a matter (litigation, investigation, regulatory inquiry) arises, the organization has a duty to **preserve** the potentially relevant content of the people involved — and to be able to **prove** it did. In Microsoft Purview that duty is met with **eDiscovery legal holds** and, for larger matters, **custodian management**: the people of interest are identified as **custodians**, their data sources (mailbox, OneDrive, the Teams and SharePoint content they touch) are placed on **hold** so nothing is purged or edited away, each custodian is issued a **legal-hold notice** they must **acknowledge**, and the whole thing is **tracked** — who is on hold, who has acknowledged, who has been reminded or escalated, and when a hold is released. This solution is a repeatable, pilot-first method to stand up that **legal-hold and custodian-management program** in Microsoft Purview eDiscovery — holds, custodian data sources, hold notifications, acknowledgment tracking, and a defensible hold register — so preservation is done consistently, evidenced, and defensible instead of ad hoc.

It delivers the **operational program**: the case + hold objects, the custodian and communication workflow, the tracking and reporting, and the pilot → validate → expand discipline. It is the program you run every

time a matter starts.

## What this is NOT — the deferred boundary and the MSEDISC boundary

- **Legal determinations are deferred to legal.** *Whether* a duty to preserve has attached, *who* is a

custodian, *what* the scope of a hold is, *when* a hold may be released, and the legal sufficiency of a hold notice are **legal/eDiscovery-counsel determinations**. This solution provides the **technical method and the evidence** — it configures the holds legal scopes, issues the notices legal drafts, and produces the tracking — it does **not** make the preservation decision or judge defensibility.

- **This is not the eDiscovery *readiness assessment* (MSEDISC).** The readiness-assessment product

assesses whether the tenant is *ready* to run eDiscovery — licensing, roles, data-source coverage, search/export posture, the maturity gaps. **This** product is the running **hold + custodian program** that operates once you are ready. Where this solution needs the readiness picture (is Premium licensed? are roles assigned? are all workloads discoverable?), it **references** MSEDISC rather than re-performing the assessment.

- **Collection, review, analytics, and export are not in scope here.** Building review sets, running

searches over held content, applying analytics/predictive coding, and producing/exporting are the *downstream* eDiscovery phases. This program stops at **defensible preservation** (hold + notice + tracking); it hands validated, preserved custodians to whatever review/production process legal runs.

- **Retention/records lifecycle is a different program.** Retention policies and records labels (keep

and dispose on a *schedule*) are the province of the data-lifecycle/records product. Legal hold is **matter-driven preservation** that *overrides* deletion for the duration of the matter — it is referenced against, not rebuilt.

## The core distinction — hold types in Microsoft Purview

The single most important design decision is **which preservation mechanism** a matter uses, because they differ in scope, evidence, and licensing:

- An **eDiscovery case hold** (a **CaseHoldPolicy** + **CaseHoldRule** created inside an eDiscovery

case) preserves the **specific locations** named on it — chosen mailboxes, OneDrive accounts, SharePoint sites, and Microsoft 365 Group/Teams locations — and can be **query-based** (preserve only content matching keywords/date range) or **preserve-everything**. It is **matter-scoped**, auditable, and the natural unit of a legal hold. This is the primary mechanism this program uses.

- **Custodian-based holds** are an **eDiscovery (Premium)** capability: you add a **custodian** (a

person of interest), Purview **maps their data sources** (primary mailbox and OneDrive, plus associated Teams/Groups and any additional mailboxes/sites you attach), and placing the custodian on hold creates the underlying case holds automatically — *and* unlocks the **communications** (hold-notice) workflow with acknowledgment tracking, reminders, and escalations. This is what turns a hold into a *managed custodian program*.

- **Litigation Hold** (`Set-Mailbox -LitigationHoldEnabled`) is a **mailbox-level** hold in Exchange

Online (Plan 2). It preserves a single mailbox (and its archive) for a duration, independent of any case. It is a useful **belt-and-suspenders** hold for a small matter or a departing custodian, but it is **not matter-scoped, has no notice/acknowledgment workflow, and only covers the mailbox** — so it complements, but does not replace, case/custodian holds.

- **Retention policies with preservation** also keep content, but they are *schedule*-driven, not matter-driven, and carry no custodian/notice workflow — referenced, not used as a legal hold here.

The reference design uses **eDiscovery case holds** as the preservation floor and, where Premium is licensed, **custodian management + communications** as the managed layer, with **Litigation Hold** as an optional mailbox-level reinforcement.

## Why this matters

- **A preservation duty that lives in an email is not preservation.** A case hold in Microsoft Purview is enforcement the platform applies — held content cannot be permanently purged even if a user deletes it or a retention policy would otherwise delete it — with an audit trail the platform keeps.
- **Defensibility is about *proof*, not just the hold.** Courts and regulators expect evidence that a hold was issued, to whom, that custodians *acknowledged* it, that non-responders were *reminded and escalated*, and that holds were *released* deliberately. The custodian-communications workflow and the hold register this program produces are that proof.
- **Spoliation is expensive and often irreversible.** If a custodian mailbox is purged, a leaver's data is deleted, or a hold is missed, the content is gone — and the sanction lands on the organization. A consistent, tracked program is the control that prevents it.
- **Ad-hoc holds do not scale.** One matter with five custodians can be run by hand; a portfolio of matters with overlapping custodians cannot. Named, standardized case/hold/notice objects, a tracked register, and scripts make it repeatable and auditable.
- **Doing it pilot-first** — proving the hold, the notice, and the tracking on a bounded pilot matter before it is the live practice — avoids the failure mode of a mis-scoped or non-distributing hold on a real matter, where the cost of getting it wrong is legal, not just technical.

## How it works (the program)

1. **Create the matter case.** In Microsoft Purview → **eDiscovery**, an eDiscovery case is created for the matter (`New-ComplianceCase`, or the portal). The case is the container for all holds, custodians, and communications for that matter.
1. **Add custodians and map their data sources** *(Premium)*. Each person of interest is added as a **custodian**; Purview resolves their **primary mailbox and OneDrive** and lets you attach **associated** Teams/Groups and **additional** mailboxes/sites. In the newer unified experience these appear under **Data sources**; verify the exact tab name in your tenant.
1. **Place the hold.** Placing custodians on hold (Premium) or creating a **case hold policy + rule**

(Standard) preserves the mapped locations. The rule can be **query-based** (a KQL condition and/or date range — preserve only matter-relevant content) or **preserve-everything**. Optionally enable **Litigation Hold** on the custodian mailboxes as reinforcement.

1. **Issue the hold notice** *(Premium communications)*. A **hold notification** legal drafts is issued

to the custodians as a **communication**; it carries the preservation instructions and an **acknowledgment** link. Purview supports **issuance**, **re-issuance/reminder**, **escalation** (to non-responders), and **release** notices, on a schedule you set.

1. **Track acknowledgment.** Each custodians **acknowledgment status** (acknowledged / not yet /

reminded / escalated) is tracked in the case; non-responders are reminded and escalated per the configured cadence. This is the evidence that the notice reached the custodian.

1. **Validate the hold is effective.** Confirm the case hold policys **distribution status is Success**

across all locations, every custodian is covered, and (where used) Litigation Hold shows enabled — with explicit pass/fail gates before the program is trusted.

1. **Maintain the register.** A **custodian hold register** — custodians, their data sources, hold

status, notice/acknowledgment status, dates — is exported and kept current as the defensible record of the matter.

1. **Release deliberately.** When legal authorizes release, the custodian is released (hold removed), a

**release notice** is sent, and the release is recorded in the register. Release is a legal decision — the program executes and evidences it, it does not decide it.

1. The whole thing runs through the house model — **Assess → Design → Configure → Pilot → Validate →

Expand **— proving the mechanics on a** pilot matter** before it becomes the live practice.

## The reference design

• **Inputs (from legal/eDiscovery counsel):** the matter, the custodian list, the preservation scope

(all content vs a query/date range), the hold-notice text and cadence (reminder/escalation intervals), and the release authority. This solution **implements** these; it does not author them.

• **Case (the container):** one eDiscovery case per matter — `Legal Hold Case – <Matter> – Pilot`.

• **Holds (the floor):** a case hold policy + rule per matter — `Hold Policy – <Matter> – Pilot` /

`Hold Rule – <Matter> – Pilot` — query-based where legal scopes it, preserve-everything where they do not.

• **Custodians (the managed layer, Premium):** custodians added to the case with mapped data sources,

grouped conceptually as `Custodians – <Matter> – Pilot`.

• **Communications (Premium):** `Hold Notice – <Matter> – Issuance`, `– Reminder`, `– Escalation`,

`– Release`, with a defined acknowledgment cadence.

- **Reinforcement (optional):** Litigation Hold enabled on custodian mailboxes for the matter duration.
- **Tracking:** `Custodian Hold Register – <Matter> – Report`, exported and kept current.
- **Scope/stage:** a **pilot matter** (`Scope – Hold-Pilot`) validated before the program is adopted as the live practice for real matters.

## What you can control

- Which **mechanism** each matter uses — case hold (Standard) vs custodian management + communications (Premium), with or without Litigation Hold reinforcement.
- Whether a hold is **query-based** (preserve only content matching a KQL/date scope legal defines) or **preserve-everything**.
- Which **data sources** each custodian contributes — primary mailbox/OneDrive plus associated Teams/Groups and additional mailboxes/sites.
- The **notice cadence** — issuance, reminder interval, escalation to non-responders, and release.
- The **pilot scope**, the pass/fail gates, and the rollback — before the program is the live practice.

## Limitations (be honest with buyers)

- **Custodian management and communications are Premium (E5-class).** Adding custodians, mapping their data sources, and the **hold-notice/acknowledgment/reminder/escalation** workflow require **eDiscovery (Premium)** — generally **Microsoft 365 E5**, the **E5 Compliance** add-on, or the **E5 eDiscovery & Audit** add-on. **eDiscovery (Standard)** case holds are available at **E3**, but *without* custodians or communications. **Verify what your tenant is licensed for** before designing the program.
- **The eDiscovery portal is actively changing.** Microsoft is consolidating the classic **eDiscovery (Standard) **and** (Premium) **experiences into a** new unified eDiscovery **experience in the Purview portal, and some tab names/paths (e.g. *Custodians* → *Data sources*) differ between them. This solution names the objects and the cmdlets, and** flags "verify the exact path in your tenant"** rather than asserting a single UI that may have moved.
- **Holds are not instant.** Placing a hold and distributing it across locations runs on service timers; **distribution status** can take time (often minutes to hours, and longer for many locations) to reach **Success** — do not judge a hold in its first minutes; validate distribution.
- **A hold does not stop a user editing or deleting *from their view*** — it stops the content from being **permanently purged**. Deleted held items are recoverable (they move to a preservation location); the hold preserves, it does not lock the item read-only the way a record label does.
- **Notice acknowledgment is a portal-tracked workflow.** Issuance, acknowledgment, reminders, and escalation are managed in the Purview eDiscovery **Communications** area; the provided scripts **inventory and report** holds, custodians, and (where the API exposes it) status, but the notice workflow itself is

operated in the portal — the scripts do not send notices.

- **Not everything is exposed to PowerShell/Graph equally.** Case holds are well-covered by Security &

Compliance PowerShell (`*-CaseHoldPolicy/Rule`); custodians and holds are readable via the **Microsoft Graph eDiscovery** APIs (some in **beta**); the **communications/acknowledgment** detail is primarily a portal surface. The scripts use only documented cmdlets/endpoints and **flag** where a step is portal-only — no invented cmdlets.

- **Commercial vs Government cloud paths differ.** Portal URLs, endpoints, and some feature availability

differ in **GCC / GCC High / DoD** (e.g. `*.office365.us`, `Connect-IPPSSession -ConnectionUri`, Graph national-cloud endpoints) — **verify the path and feature in your cloud.**

## Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Matter/Scope> – <Stage>` and reused verbatim across the docs, scripts, and the Purview admin surfaces (`Contoso-Matter-001` is used as the placeholder matter throughout):

| Artifact | Name |
|---|---|
| eDiscovery case (the matter) | `Legal Hold Case – Contoso-Matter-001 – Pilot` |
| Case hold policy | `Hold Policy – Contoso-Matter-001 – Pilot` |
| Case hold rule | `Hold Rule – Contoso-Matter-001 – Pilot` |
| Custodian set (conceptual grouping) | `Custodians – Contoso-Matter-001 – Pilot` |
| Hold notice — issuance | `Hold Notice – Contoso-Matter-001 – Issuance` |
| Hold notice — reminder | `Hold Notice – Contoso-Matter-001 – Reminder` |
| Hold notice — escalation | `Hold Notice – Contoso-Matter-001 – Escalation` |
| Hold notice — release | `Hold Notice – Contoso-Matter-001 – Release` |
| Pilot scope | `Scope – Hold-Pilot` |
| Hold register (tracking export) | `Custodian Hold Register – Contoso-Matter-001 – Report` |
| Preservation posture (discovery export) | `Hold Posture – Tenant – Discovery` |

When the pilot passes and the program goes live on real matters, mint the production equivalents by swapping the `– Pilot` stage for the real matter name/number (e.g. `Hold Policy – <RealMatter> – Live`) — never edit a validated pilot artifact in place, and never reuse a pilot case for a real matter.

### The standard deployment model

> ***Assess*** *the tenants preservation posture and eDiscovery readiness (reference MSEDISC), and ingest the matter/custodian/scope inputs from legal →* ***Design*** *the case/hold/custodian/notice map →* ***Configure*** *the case, case hold policy + rule, custodians and data sources, and the notice cadence →* ***Pilot*** *on* `Scope – Hold-Pilot` *(a pilot matter with test custodians; holds created with* `-WhatIf` *reviewed first, then committed) →* ***Validate*** *distribution status, custodian coverage, and notice/acknowledgment against explicit pass/fail gates →* ***Expand*** *into the live matter practice with the production naming and change control.*

Preservation is **high-stakes** — a missed custodian, a non-distributing hold, or a released hold that should have stayed is a legal exposure, not just a technical one. Every real-matter action goes through change control with the register updated, and nothing becomes the live practice until the gates in Section 5 pass on the pilot. **Legal owns the determination; this program owns the execution and the evidence.**

# 2. Requirements — Legal-Hold & Custodian Management Program (Microsoft Purview)

These are the **solution-specific** prerequisites for standing up a legal-hold and custodian-management program in **Microsoft Purview eDiscovery** — cases, case holds, custodians, hold notifications, and tracking. They build on — and do not repeat — the six shared prerequisites in `global-requirements-purview.md` (tenant + compliance licensing, least-privilege Purview roles, S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control). Verify each item below before starting the Implementation Guide.

For every requirement: **Description** (what it is), **Why required**, **How to verify**, **How to implement if missing**.

## 2.1 Licensing sufficient for the eDiscovery tier you will use — Standard vs Premium

- **Description:** The program uses two eDiscovery tiers, and they are licensed differently.

**eDiscovery (Standard)** — cases and **case holds** (`CaseHoldPolicy` + `CaseHoldRule`) — is broadly available at **Microsoft 365 E3** (Exchange Online Plan 2 / SharePoint Online Plan 2). **eDiscovery (Premium)** — **custodian management**, **custodian data-source mapping**, and the **hold-notice (communications) / acknowledgment / reminder / escalation** workflow — requires **Microsoft 365 E5**, the **E5 Compliance** add-on, or the **E5 eDiscovery & Audit** add-on. Per-custodian Premium licensing applies to each user placed under custodian management. This extends GR-#1 with the eDiscovery-specific plan detail.

- **Why required:** The custodian program's defining features — mapped data sources and the tracked hold

notice — are **Premium-only**. On an E3-only tenant you can still run **case holds** (the preservation floor), but not custodians or communications; designing for a feature the tenant is not licensed for is the most common early failure.

• **How to verify:** In the **Microsoft 365 admin center** confirm the subscription/add-ons (E3 / E5 / E5

Compliance / E5 eDiscovery & Audit). In the **Purview portal** → **eDiscovery**, confirm whether the **Premium** experience (Custodians / Data sources, Communications) is present, not just **Standard**. **Flag "verify your licensing/tenant"** — tiers and the unified-eDiscovery rollout evolve.

• **How to implement if missing:** Acquire/assign the Premium licensing (and per-custodian licenses) before

building the custodian/communications layer; on E3, scope the program to **case holds** and operate the notice workflow manually until Premium is licensed.

## 2.2 eDiscovery least-privilege role-group membership

• **Description:** eDiscovery in Purview is gated by **role groups**, not general compliance roles.

**eDiscovery Manager** — create and manage **your own** cases, holds, and (Premium) custodians/ communications; **eDiscovery Administrator** — a member of the eDiscovery Manager group who can additionally **see and manage all cases** in the tenant (the program owner); **Reviewer** and **Compliance Data Administrator** for narrower read/analysis roles. The underlying roles (Case Management, Hold, Custodian, Communication, Review, Search And Purge, Export) are assigned **through** these groups. Global Administrator only where genuinely required. Extends GR-#2.

• **Why required:** A user without eDiscovery role-group membership cannot see the eDiscovery node at all; a

plain eDiscovery Manager sees only cases they are a member of. Running a tenant-wide **program** (many matters, oversight, reporting) needs at least one **eDiscovery Administrator**, with least-privilege Managers per matter — and separation keeps the blast radius small on a preservation-critical control.

• **How to verify:** **Purview portal → Settings → Roles & scopes → Permissions (Microsoft Purview

roles) → **confirm** eDiscovery Manager / eDiscovery Administrator** membership; or `Get-eDiscoveryCaseAdmin` / `Get-RoleGroupMember 'eDiscovery Manager'` in S&C PowerShell.

• **How to implement if missing:** Assign the least-privilege eDiscovery role group under change control —

one or more **eDiscovery Administrators** as program owners, **eDiscovery Managers** scoped per matter; separate the person who *places* holds from the person who *authorizes release* where possible.

## 2.3 In-scope custodians, data sources, and workloads identified (by legal, implemented here)

• **Description:** A per-matter list of the **custodians** (people of interest) and the **data sources** to

preserve — primary **mailbox** and **OneDrive**, plus **associated** Teams/Microsoft 365 Groups and any **additional** mailboxes/SharePoint sites — and **non-custodial** locations (shared mailboxes, departmental sites, records repositories) that must be held but belong to no single person. Extends GR-#4.

• **Why required:** A case hold and a custodian mapping key off *specific* locations; a missed custodian or an

un-mapped associated site is content that is **not preserved** — a spoliation exposure, not a cosmetic gap. Non-custodial data (a team SharePoint site, a shared mailbox) is invisible to custodian mapping and must be held explicitly.

- **How to verify:** Confirm the custodian list and each custodian's mailbox/OneDrive resolve

(`Get-Recipient`); confirm associated Teams/Groups and additional sites with the custodian and IT; confirm the non-custodial locations with the data owners. **The *who* and *what-scope* are legal's determination** — this solution implements and evidences them.

- **How to implement if missing:** Capture the custodian/data-source/non-custodial map from legal before

configuring; the Discovery and Set-Scope scripts inventory what a hold currently covers so gaps are visible.

## 2.4 Preservation scope and hold-notice content defined by legal (drafted by legal, configured here)

- **Description:** For each matter, legal defines: the **preservation scope** — **preserve everything** vs a

**query-based** hold (a KQL condition and/or date range that preserves only matter-relevant content); the **hold-notice text** (Premium communications) — issuance, reminder, escalation, and release wording; and the **acknowledgment cadence** (reminder interval, escalation trigger for non-responders). Extends GR-#5.

- **Why required:** The hold rule's `ContentMatchQuery`/date range and the notice's legal sufficiency are

**legal determinations**. A query-based hold that is too narrow silently fails to preserve responsive content; a notice that is legally insufficient undermines defensibility. The program **configures** these values — it does not author them.

- **How to verify:** Confirm the preservation scope (everything vs query/date), the four notice texts, and the

cadence with eDiscovery counsel; confirm any query with a small **statistics** run before it becomes a hold.

- **How to implement if missing:** Have legal supply the scope and notice content; default to

**preserve-everything** (the safest floor) where a query has not been authorized, never the reverse.

## 2.5 A named release authority and change-control path (holds are preservation-critical)

- **Description:** A named **release authority** (the legal/eDiscovery role empowered to authorize lifting a

hold) and a **change-control record** for every hold placement, scope change, and release. Because releasing a hold **deletes the preservation** and cannot be undone for content purged afterward, release must be a deliberate, signed-off, two-party action — **never** scripted automatically. Extends GR-#6.

- **Why required:** The single highest-risk action in this program is **releasing a hold that should have

stayed**. A named authority, a written authorization, and a change record are what make placement and release defensible — and what this solution's tooling deliberately** will not** automate (no script in this pack releases a hold).

• **How to verify:** Confirm who may authorize release, the written-authorization format, and that the release

step is outside the automation (portal + sign-off only).

• **How to implement if missing:** Name the release authority and stand up the change log before any hold is

placed; wire the sign-off into the operating procedure, not the scripts.

## 2.6 Audit logging enabled and a pilot matter defined

• **Description:** Tenant **unified audit logging** enabled (so hold placement, notice, and access events are

recorded and reportable), and a **pilot matter** — a bounded case with **fabricated test custodians** — on which the mechanics are proven before the program runs on a real matter (`Scope – Hold-Pilot`, the case `Legal Hold Case – Contoso-Matter-001 – Pilot`). Extends GR-#6.

• **Why required:** Defensibility rests on **evidence**, and the evidence lives in the audit log and the case

history — if auditing is off, the tracking report has nothing to read. The pilot proves the hold distributes, the notice issues, and acknowledgment tracks **before** a real matter's preservation depends on it — the house model's *pilot-first* discipline applied to a control where getting it wrong is a legal, not a technical, cost.

• **How to verify:** `Get-AdminAuditLogConfig`/audit-log search confirms auditing is on; confirm the pilot

case name, the fabricated test custodians, and the change window to expand to real matters.

• **How to implement if missing:** Enable unified audit logging; define the pilot matter and test custodians

(never real people) before validating.

### *Requirements summary*

| # | Requirement | Gate |
|---|---|---|
| GR 1–6 | Category prerequisites (`global-requirements-purview.md`) | Tenant/licensing, roles, toolchain, scope, stakeholders, pilot + change control |
| 2.1 | eDiscovery licensing — Standard (E3) vs Premium (E5/E5 Compliance/E5 eDiscovery & Audit); verify | Custodians + communications available (Premium) |
| 2.2 | eDiscovery role-group membership (Manager / Administrator) | Case, hold, custodian, communication access |
| 2.3 | In-scope custodians, data sources, non-custodial locations | Nothing responsive left un-held |
| 2.4 | Preservation scope + hold-notice content (from legal) | Hold rule + notice configured to legal's intent |

| # | Requirement | Gate |
|---|---|---|
| 2.5 | Named release authority + change control | Release is deliberate, signed, never scripted |
| 2.6 | Audit logging on + pilot matter with test custodians | Evidence captured; mechanics proven pilot-first |

> ***Deferred to legal / eDiscovery counsel / records:*** *whether a duty to preserve has attached, *who* is a custodian, the *scope* of preservation, the legal sufficiency of a notice, and *when* a hold may be released. This solution provides the technical configuration and the evidence, not the determination — and it never releases a hold automatically.*

# Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

## 1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

## 2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

## 3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

## 4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

## 5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

## 6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

# 3. Implementation Guide — Legal-Hold & Custodian Management Program (Microsoft Purview)

This is the exact, ordered procedure to stand up a legal-hold and custodian-management program in **Microsoft Purview eDiscovery** — case, case hold, custodians, hold notifications, and tracking — proven on a **pilot matter** with **fabricated test custodians** first, then adopted as the live practice for real matters. Portal paths and cmdlets are from Microsoft's public documentation; **licensing-/plan-dependent features and evolving portal paths are flagged "verify your licensing/tenant"** and are never asserted for your tenant. Do the phases in order. Each phase names its artifacts (from the Overview naming table), states a **pass/fail gate**, and gives a **rollback**.

> ***Automation.*** *The five delivered PowerShell tools run through **Security & Compliance PowerShell** (`Connect-IPPSSession`) and **Microsoft Graph** (eDiscovery, some endpoints **beta**). They are **read-only or** `-WhatIf` **by default**; writes require an explicit `-Commit` switch. **No script in this pack releases a hold** — release is a deliberate portal + sign-off action. No credentials are stored in code.*
>
> ***Determinations deferred.*** **Whether* a duty to preserve has attached, *who* is a custodian, the *scope* of the hold, the legal sufficiency of a notice, and *when* a hold may be released are **legal / eDiscovery- counsel** decisions. This guide configures and proves the control; capture those decisions before Phase 3.*
>
> ***Scriptability caveat. Case holds*** *(`*-CaseHoldPolicy` / `*-CaseHoldRule`) and cases (`*-ComplianceCase`) are well covered by **Security & Compliance PowerShell**. **Custodians** and holds are readable/creatable via the **Microsoft Graph eDiscovery** APIs (some in **beta**). The **communications / hold-notice / acknowledgment** workflow is primarily a **portal** surface — the scripts **inventory and report** it where the API exposes status, but issuing/acknowledging notices is done in the Purview portal. Where a step is portal-only it is flagged; no cmdlet is invented.*

## Phase 0 — Connect and baseline (Assess)

1. **Connect the toolchain** (GR-#3). On the admin workstation:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force   # provides Connect-IPPSSession
Connect-IPPSSession -UserPrincipalName admin@contoso.com            # Security & Compliance PowerShell
# Graph (eDiscovery custodian/case read) — beta scopes, verify availability in your tenant:
Connect-MgGraph -Scopes 'eDiscovery.Read.All','Organization.Read.All' -NoWelcome
```

1. **Run the posture assessment** (delivered script #1, read-only):

```
.\EDHOLD-Discovery.ps1 -CsvPath .\EDHOLD-Posture.csv
```

It reports existing eDiscovery cases, case hold policies and their **distribution status**, any existing custodian/communications signal (via Graph, where licensed), a **Premium-licensing signal**, and eDiscovery role-group membership.

1. **Confirm licensing and roles** (Req 2.1, 2.2): confirm Standard vs Premium and that at least one

**eDiscovery Administrator** owns the program, with **eDiscovery Managers** per matter.

**Pass/fail gate:** you have a written baseline (existing cases, holds and their distribution status, role membership) and a confirmed licensing tier (Standard = case holds only; Premium = custodians + communications). **Rollback:** none (read-only).

## Phase 1 — Design the matter case and preservation scope (Design)

Take the matter, custodian list, and preservation scope **from legal** (Req 2.3/2.4) and design the objects. Do **not** invent a preservation scope — default to preserve-everything unless legal has authorized a query.

1. **Create the pilot case.** In **Purview portal** → **eDiscovery** (verify Standard vs the new unified

experience in your tenant), create the case `Legal Hold Case – Contoso-Matter-001 – Pilot`, or:

```
New-ComplianceCase -Name 'Legal Hold Case – Contoso-Matter-001 – Pilot' `
    -CaseType eDiscovery -Description 'Pilot matter — fabricated custodians only'
```

1. **Design the hold.** Decide, with legal, whether the hold is **preserve-everything** or **query-based** (a

`ContentMatchQuery` KQL condition and/or date range). For a query-based hold, validate the query with a **statistics/estimate** run in the portal *before* it becomes a hold, so you know it captures responsive content. *The query and date range are legal's determination — configured here, not authored here.*

1. **Design the custodian and non-custodial map** *(Premium for custodians)*. List each custodian and their

data sources (primary mailbox/OneDrive + associated Teams/Groups + additional mailboxes/sites), and the **non-custodial** locations (shared mailboxes, departmental sites) to hold separately. Group conceptually as `Custodians – Contoso-Matter-001 – Pilot`.

1. **Design the notice cadence** *(Premium communications)*. With legal, fix the four notices —

`Hold Notice – Contoso-Matter-001 – Issuance` / `– Reminder` / `– Escalation` / `– Release` — and the acknowledgment cadence (reminder interval, escalation trigger).

**Pass/fail gate:** a written case/hold/custodian/notice design exists, the preservation scope is chosen (everything vs a validated query), and every custodian's data sources and every non-custodial location are named. **Rollback:** design only — nothing configured yet; discard the design.

## Phase 2 — Configure the case hold in the pilot (Configure — preview first)

Build the preservation **floor** — the case hold policy + rule — reviewing the intended change with `-WhatIf` before committing. This is the mechanism that works at **every** tier (Standard and Premium).

1. **Create the case hold policy** naming the locations, scoped to the **pilot/test** locations only. Preview,

then commit:

```
.\EDHOLD-Config.ps1 -Action NewCaseHold `
    -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' `
    -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' `
    -ExchangeLocation 'testcustodian1@contoso.com' -SharePointLocation 'https://contoso.sharepoint.com/
    -WhatIf
# add -Commit to apply (creates New-CaseHoldPolicy)
```

The helper wraps `New-CaseHoldPolicy -Case <case> -ExchangeLocation ... -SharePointLocation ...`.

1. **Create the case hold rule.** A **preserve-everything** rule holds all content in the policy's locations;

a **query-based** rule adds legal's `ContentMatchQuery` / date range:

```
.\EDHOLD-Config.ps1 -Action NewCaseHoldRule `
    -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' `
    -HoldRuleName 'Hold Rule – Contoso-Matter-001 – Pilot' `
    -ContentMatchQuery '' -WhatIf      # empty query = preserve everything
# add -Commit to apply (creates New-CaseHoldRule)
```

The helper wraps `New-CaseHoldRule -Name <rule> -Policy <policy> [-ContentMatchQuery <kql>]`.

1. **(Optional) Litigation Hold reinforcement.** For a small matter or a departing custodian, enable a

mailbox-level hold as belt-and-suspenders — a deliberate, separately-tracked action:

```
.\EDHOLD-Config.ps1 -Action EnableLitigationHold -ExchangeLocation 'testcustodian1@contoso.com' `
    -LitigationHoldDurationDays 3650 -WhatIf      # add -Commit to apply Set-Mailbox -LitigationHoldEn
```

**Pass/fail gate:** the case hold policy + rule exist on the pilot case, scoped to the **test** locations, with the intended preservation scope (everything vs the validated query); `-WhatIf` output was reviewed before `-Commit`. **Rollback:** remove the pilot hold — `Remove-CaseHoldPolicy`/`Remove-CaseHoldRule` in the portal or via the helper's `-Action RemoveCaseHold -WhatIf` (preview first). Removing a **pilot** hold on test custodians is safe; **never** remove a hold on a real matter without the release authority's sign-off (Req 2.5).

## Phase 3 — Add custodians and issue the hold notice (Configure — Premium)

*(Premium only. On E3/Standard, skip to Phase 4 and operate the notice workflow manually; the case hold from Phase 2 is your preservation.)* Capture legal's custodian, scope, and notice decisions (Req 2.4) **before** this phase.

1. **Add custodians and map data sources.** In **Purview portal → eDiscovery (Premium) → the case →

Data sources / Custodians **(tab name differs between the classic and new unified experience —** verify in your tenant**), add each** test **custodian; Purview resolves their primary mailbox/OneDrive and lets**

**you attach associated Teams/Groups and additional mailboxes/sites. Placing a custodian** on hold **creates the underlying case holds automatically. *Custodian creation is a Graph-eDiscovery / portal action; the** `EDHOLD-Config.ps1 -Action AddCustodian` **path uses the Graph eDiscovery API where licensed and** beta** — if unavailable in your tenant, add custodians in the portal.*

1. **Add non-custodial data sources.** Attach the shared mailboxes/departmental sites that belong to no custodian as **non-custodial data sources** on the case (portal), so they are held explicitly.

1. **Issue the hold notice** *(portal Communications)*. In the case **Communications**, create `Hold Notice – Contoso-Matter-001 – Issuance` from legal's text with an **acknowledgment** requirement, and configure the **reminder** (`– Reminder`) and **escalation** (`– Escalation`) on legal's cadence. **Issue** it to the test custodians. *Issuing/acknowledging notices is a portal workflow — no cmdlet sends a notice; the scripts report status only.*

**Pass/fail gate:** every test custodian is added with mapped data sources and shows **on hold**; non-custodial locations are held; the issuance notice is issued to the test custodians with reminder/escalation configured. **Rollback:** remove a **test** custodian / release the pilot notice **in the portal**. On a real matter, custodian removal or notice release follows the release authority's sign-off (Req 2.5) — it is never scripted.

## Phase 4 — Validate the hold and the notice (Validate)

Confirm the preservation is actually effective **before** the program is trusted — distribution succeeded, every custodian/location is covered, and notices issued and are being acknowledged.

1. **Confirm hold distribution.** Placing a hold runs on service timers; **distribution status** can take minutes to hours (longer for many locations) to reach **Success** — do not judge a hold in its first minutes. Validate with the delivered script (read-only):

```
.\EDHOLD-Validate.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' `
    -ExpectedExchangeLocation 'testcustodian1@contoso.com' -Verbose
```

It reports per-gate pass/fail: hold **distribution = Success** across locations, expected custodians/ locations in scope, Litigation Hold state (where used), and (via Graph, where exposed) notice/acknowledgment signal.

1. **Prove preservation.** With fabricated content, confirm a **hard-deleted** item on a held test mailbox is recoverable (moves to the preservation/Recoverable Items location) — the hold prevents **permanent purge**; it does not lock the item read-only. Record the evidence.

1. **Confirm acknowledgment tracking** *(Premium)*. In **Communications**, confirm each test custodian's acknowledgment status is tracked (acknowledged / not yet / reminded / escalated) and that a non-responder is reminded/escalated per cadence.

**Pass/fail gate:** distribution = **Success** on all pilot locations, every expected custodian/location is covered, deleted held content is recoverable, and (Premium) notices issued with acknowledgment tracked — all against explicit pass/fail gates (Section 5). **Rollback:** none (read-only validation); if a gate fails, fix the scope/config in Phase 2/3 and re-validate — do **not** proceed to real matters on a failed gate.

## Phase 5 — Track, then expand to live matters (Operate → Expand)

1. **Produce the hold register.** Export the defensible tracking record (read-only):

```
.\EDHOLD-Report.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' `
    -CsvPath '.\Custodian Hold Register – Contoso-Matter-001 – Report.csv'
```

It produces `Custodian Hold Register – Contoso-Matter-001 – Report` — custodians, data sources, hold/ distribution status, and (where exposed) notice/acknowledgment status and dates — the record legal relies on.

1. **Maintain scope on change.** As a custodian's role or data sources change, adjust **hold locations / custodian data sources **with the scope helper (preview first) — it** adds/adjusts**, it never releases:

```
.\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' `
    -AddExchangeLocation 'testcustodian2@contoso.com' -WhatIf   # add -Commit to apply
```

1. **Expand to the live practice.** Once the pilot gates pass, mint the **production** equivalents for real matters by swapping the `– Pilot` stage for the real matter name (e.g. `Legal Hold Case – <RealMatter> – Live`, `Hold Policy – <RealMatter> – Live`) — **never** edit a validated pilot artifact in place, and **never** reuse the pilot case for a real matter. Each real-matter hold goes through change control (Req 2.5) with the register updated.

1. **Release deliberately (portal + sign-off only).** When the **release authority** authorizes it in writing, release the custodian / remove the hold **in the portal**, issue the `Hold Notice – <Matter> – Release`, and record the release in the register. **No script in this pack releases a hold** — release is a two-party, signed, non-automated action. Confirm no *other* matter still requires the same location before releasing (overlapping holds).

**Pass/fail gate:** the hold register is exported and current; scope changes are additive and tracked; real matters use production naming under change control; releases are signed, evidenced, and never scripted. **Rollback:** a mis-scoped **addition** can be reverted (remove the added location, preview first); a **release** cannot be undone for content purged afterward — which is exactly why it is gated behind sign-off, not automation.

## Order-of-operations summary

| Phase | Do | Artifact(s) | Gate | Rollback |
|---|---|---|---|---|
| 0 Assess | Connect, baseline cases/holds/roles, confirm licensing tier | Posture CSV | Baseline + tier confirmed | n/a (read-only) |
| 1 Design | Create pilot case; design hold scope, custodian/notice map (from legal) | Pilot case + written design | Case + design complete | Discard design |
| 2 Configure | Build case hold policy + rule (preview → commit); optional Lit Hold | Hold policy + rule | Hold scoped to test locations | Remove pilot hold (preview) |

| Phase | Do | Artifact(s) | Gate | Rollback |
|---|---|---|---|---|
| 3 Configure (Premium) | Add test custodians + non-custodial sources; issue notice (portal) | Custodians, notices | Custodians on hold, notice issued | Remove test custodian / release pilot notice (portal) |
| 4 Validate | Confirm distribution=Success, coverage, preservation, acknowledgment | Validation report | All gates pass | Fix + re-validate (never proceed on fail) |
| 5 Operate → Expand | Export register, maintain scope, go live on real matters, release by sign-off | Hold register | Register current; releases signed, never scripted | Revert additions; release is not reversible → gated by sign-off |

# 5. Testing & Validation — Legal-Hold & Custodian Management Program (Microsoft Purview)

Validate the legal-hold and custodian-management program against explicit **pass/fail gates**. The three headline gates are: **(A) the hold actually preserves**, **(B) custodians and notices are covered and tracked**, and **(C) the program is operable and defensible**. Run **every** test on the **pilot matter** `Legal Hold Case – Contoso-Matter-001 – Pilot` with **fabricated test custodians and content only** — never real people, real matters, or real data. Record the evidence.

> *All tests assume the case, case hold, and (Premium) custodians/notices from the Implementation Guide, proven on the pilot matter first. Custodian/communications tests require **eDiscovery (Premium)** (Req 2.1); on Standard, run Gate A and the Standard rows of Gate C, and operate the notice workflow manually.*
>
> ***Preservation-critical.** These tests validate that holds work **before** a real matter's preservation depends on them. **No test releases a hold on a real matter.** The only "release" exercised here is removing a **pilot** hold on **test** custodians (S2), which is safe by construction.*

## Test A — Preservation is effective (the hold actually holds)

| # | Test | Steps | Pass condition |
|---|---|---|---|
| A1 | Hold distribution succeeds | After placing the pilot hold, run `EDHOLD-Validate.ps1 -CaseName '…Pilot'` and check the case hold policy's distribution status. | Distribution status = **Success** across every held location (allow service-timer latency — minutes to hours before judging). |

| # | Test | Steps | Pass condition |
|---|---|---|---|
| A2 | Deleted content is preserved | On a **held** test mailbox, hard-delete a fabricated item (empty Deleted Items / purge). | The item is **recoverable** from the preservation/Recoverable Items location — the hold prevented permanent purge. |
| A3 | Query-based hold captures the right scope | If the pilot uses a query-based rule, place fabricated content **inside** and **outside** the `ContentMatchQuery`/date scope. | In-scope content is preserved; out-of-scope content behaves per policy — the query scope matches legal's intent. |
| A4 | Every intended location is on hold | Compare the held locations to the design (Req 2.3) via `EDHOLD-Validate.ps1 -ExpectedExchangeLocation` …. | No intended custodian mailbox / OneDrive / site / non-custodial location is missing from the hold. |
| A5 | Litigation Hold reinforcement (if used) | Where a test mailbox has Litigation Hold enabled, confirm state. | `LitigationHoldEnabled = True` with the intended duration — the belt-and-suspenders hold is live. |

**Gate A (pass):** distribution = Success on all pilot locations, deleted held content is recoverable, the query scope (if used) matches intent, and no intended location is missing.

## Test B — Custodians & notices are covered and tracked (Premium)

| # | Test | Steps | Pass condition |
|---|---|---|---|
| B1 | Custodian data-source mapping | For each **test** custodian, confirm the mapped data sources (primary mailbox/OneDrive + associated Teams/Groups + additional sites). | Each custodian shows **on hold** with the expected data sources mapped — nothing responsive un-mapped. |
| B2 | Non-custodial data held | Confirm the non-custodial locations (shared mailbox / departmental site) are held on the case. | Each non-custodial location is preserved — content belonging to no single person is not missed. |

| # | Test | Steps | Pass condition |
|---|---|---|---|
| B3 | Notice issued | Confirm `Hold Notice – Contoso-Matter-001 – Issuance` was issued to the test custodians in **Communications**. | Every test custodian received the issuance notice with an acknowledgment requirement. |
| B4 | Acknowledgment tracked | Acknowledge as one test custodian; leave another un-acknowledged. | The case tracks **acknowledged** vs **not yet**; the un-acknowledged custodian is eligible for reminder/escalation. |
| B5 | Reminder / escalation cadence | Advance (or simulate) the cadence for the non-responder. | A **reminder** (`–Reminder`) and then **escalation** (`–Escalation`) fire per legal's configured cadence. |
| B6 | Register reflects reality | Run `EDHOLD-Report.ps1 -CaseName '…Pilot' -CsvPath …`. | The exported `Custodian Hold Register – … –Report` matches the portal state (custodians, hold status, notice/ack status, dates). |

**Gate B (pass):** every test custodian on hold with mapped sources, non-custodial locations held, notice issued, acknowledgment/reminder/escalation tracked, and the register faithfully reflects the case.

## Test C — Operable & defensible (the control can be run and proven)

| # | Test | Steps | Pass condition |
|---|---|---|---|
| C1 | Evidence trail present | Confirm hold-placement, notice, and access events are in the case history / unified audit log (Req 2.6). | A complete, exportable evidence trail exists (who placed the hold, when; who was noticed; who acknowledged). |
| C2 | Role separation holds | Confirm who can see/manage the case and who can authorize release. | Least-privilege eDiscovery roles hold (Req 2.2); the **release authority** is distinct and named (Req 2.5). |

| # | Test | Steps | Pass condition |
|---|---|---|---|
| C3 | Overlapping-hold safety | Place a fabricated location under **two** pilot holds; release one. | The location stays preserved by the remaining hold — releasing one matter does not un-preserve content another still holds. |
| C4 | Register is defensible | Review the exported register as the record of the matter. | The register answers "who is on hold, since when, noticed, acknowledged, released?" without reconstructing from memory. |
| C5 | Release is deliberate, not scripted | Confirm the release path. | Release happens **only** in the portal with the release authority's written sign-off — **no script releases a hold** (verify the pack has no release automation). |

**Gate C (pass):** the evidence trail is complete and exportable, roles/release authority are separated and named, overlapping holds are safe, the register is defensible, and release is a signed portal action with no automation.

## Simulation-first & safety validation

| # | Test | Pass condition |
|---|---|---|
| S1 | Pilot isolation | All tests run on `Legal Hold Case – Contoso-Matter-001 – Pilot` with **fabricated** custodians/content — no real matter or person is touched. |
| S2 | Safe pilot teardown | Removing the **pilot** hold on **test** custodians (portal or `EDHOLD-Config.ps1 -Action RemoveCaseHold`, previewed with `-WhatIf`) cleanly removes only the pilot preservation — real matters are unaffected. |

| # | Test | Pass condition |
|---|---|---|
| S3 | Scripts are safe-by-default | `EDHOLD-Config.ps1` and `EDHOLD-Set-Scope.ps1` run **without** `-Commit` make no change (`-WhatIf` output only); Discovery / Validate / Report are read-only; **none** releases a real-matter hold. |
| S4 | No auto-release | Confirm no delivered script performs a hold **release** on any non-pilot case — release is portal + sign-off only (Req 2.5). |

### Evidence to capture (for the deliverable record)

• Baseline posture CSV (Phase 0): existing cases, holds and distribution status, role membership.

• Distribution = Success screenshot/log across pilot locations (Gate A1) and the deleted-item recovery evidence (A2).

• Custodian data-source mapping and non-custodial hold confirmation (B1/B2).

• Notice issuance + acknowledgment/reminder/escalation tracking (B3–B5) and the exported hold register (B6).

• The audit/evidence trail (C1), the role/release-authority separation (C2), and the overlapping-hold safety result (C3).

• Confirmation of safe pilot teardown (S2) and that no script releases a real-matter hold (S3/S4).

> ***Determination boundary.*** *These tests prove the* ***program works and is defensible****. *Whether* a given custodian, preservation scope, notice, or release is legally sufficient is an* ***eDiscovery-counsel / legal*** *sign-off, captured alongside the evidence above. This solution proves the mechanics; legal owns the determination.*

## Troubleshooting — Legal-Hold & Custodian Management Program

Work the symptom to the most specific cause below. All checks are read-only; never release or delete a hold to "test" — use `EDHOLD-Validate.ps1` and the portal to confirm state. Legal-hold *scope and release* decisions are the legal team's; this guide covers the technical configuration only.

### The hold saves but content isn't preserved

• **Symptom:** a `Legal Hold Case – <Matter> – Pilot` case hold shows created, but a custodian's edited/deleted

items are still gone.

- **Most likely:** the hold policy is **created but not started/enabled**, or the location wasn't added to the

hold rule. Run `EDHOLD-Validate.ps1` — it reports each `*-CaseHoldPolicy`/`*-CaseHoldRule`'s **Enabled/Distribution** state and the bound locations.

- **Fix:** ensure the case-hold **policy is enabled** and the rule includes the custodian's mailbox/site;

re-run distribution. Hold takes effect only after distribution reaches **Success**.

## "Distribution pending / partial" never completes

- **Cause:** large custodian counts, a location that no longer exists (deleted mailbox/site), or a transient

service delay. Distribution is asynchronous and can take time.

- **Fix:** wait and re-check `Get-CaseHoldPolicy -Identity <name> -DistributionDetail` (surfaced by

`EDHOLD-Report.ps1`). Remove/replace invalid locations, then redistribute. Escalate to Microsoft only after a location is confirmed valid and distribution stays failed.

## Can't add custodians / custodian communications missing

- **Cause:** custodian management + hold notifications are **eDiscovery (Premium)** features; on

**Standard** they aren't available, or the account lacks the **eDiscovery Manager** role.

- **Fix:** confirm licensing (`EDHOLD-Discovery.ps1` flags Standard vs Premium — *verify in your tenant*) and

role-group membership. Custodian/communications operations route through the Graph eDiscovery (beta) surface or the portal; the scripts never invent a cmdlet for them.

## "Access denied" creating or viewing a case/hold

- **Cause:** the operator isn't in **eDiscovery Manager** (create/manage own cases) or **eDiscovery

Administrator** (see all cases), or the case membership doesn't include them.

- **Fix:** add the operator to the correct role group and to the case's members. Verify with

`EDHOLD-Discovery.ps1`.

## Overlapping holds — worried about releasing preserved data

- **Behaviour:** a location under **multiple** holds stays preserved until **all** holds on it are removed.

Removing one case hold does **not** release content still held by another (or by a retention policy / Litigation Hold).

- **Fix:** before any release, run `EDHOLD-Validate.ps1` to enumerate every hold touching the location.

Release is a **legal decision** — the scripts never auto-release (the teardown path is pilot-only, guarded, and preview-first).

### Litigation Hold vs case hold confusion

- **Litigation Hold** (`Set-Mailbox -LitigationHoldEnabled`) holds an entire mailbox; **case holds** are

query/matter-scoped across mailboxes and sites. Use case holds for a specific matter; don't rely on one to do the other. `EDHOLD-Discovery.ps1` reports both.

### Pilot cleanup left a hold in place

- **Cause:** teardown skipped, or a hold was still distributing when cleanup ran.
- **Fix:** confirm the matter is closed with **legal sign-off**, then remove the pilot case holds

(`Remove-CaseHoldPolicy`, guarded/preview-first) — never as an untracked ad-hoc action. Record the release authority in the **Custodian Hold Register**.

### Government / sovereign cloud

- Portal URLs and connection endpoints differ on **GCC / GCC High / DoD** (`-ConnectionUri` / Graph national

cloud host). Confirm the correct endpoint for your cloud before running — *verify against current Microsoft guidance*; do not assume commercial paths.

*If holds are enabled, distributed to **Success**, custodians tracked (Premium), and validation passes but a matter still can't produce expected content, the issue is scope/query design (a legal + eDiscovery question), not a preservation fault — engage the eDiscovery/legal owners rather than changing the hold.*

## 7. Best Practices — Legal-Hold & Custodian Management Program (Microsoft Purview)

Field-tested practices for running legal holds and custodians in Microsoft Purview eDiscovery so preservation is consistent, tracked, and **defensible** — and so the one irreversible action (releasing a hold) is never taken by accident. These complement the Implementation and Testing guides.

### Prove it on a pilot matter before a real one depends on it

- **Never** learn the mechanics on a live matter. Run the full flow — case, hold, custodians, notice,

tracking — on `Legal Hold Case – Contoso-Matter-001 – Pilot` with **fabricated** custodians first (Req 2.6), and pass the Section 5 gates before the program is the live practice.

- Mint **production** artifacts for real matters (`… – <RealMatter> – Live`); **never** edit a validated pilot

artifact in place and **never** reuse the pilot case for a real matter.

- Treat the pilot as a **repeatable template**, not a one-off — the value is a program you run the same way

every time a matter starts.

## Preserve first, scope later — default to everything

• When in doubt, **preserve-everything** is the safe floor; a **query-based** hold that is too narrow silently fails to preserve responsive content. Only narrow to a `ContentMatchQuery`/date range when **legal has authorized it**, and validate the query with a **statistics/estimate** run before it becomes a hold.

• A missed custodian or an un-mapped associated site is **un-preserved content** — enumerate primary mailbox/OneDrive **and** associated Teams/Groups and additional mailboxes/sites, and hold **non-custodial** locations (shared mailboxes, departmental sites) explicitly.

## Validate distribution — a hold placed is not a hold effective

• Placing a hold runs on **service timers**; **distribution status** can take minutes to hours (longer for many locations) to reach **Success**. Do not judge a hold in its first minutes — **validate distribution** with `EDHOLD-Validate.ps1` before declaring a custodian preserved.

• Confirm the hold actually preserves by proving a **hard-deleted** test item is recoverable — the hold prevents **permanent purge**, it does not lock the item read-only the way a record label does.

## Notice, acknowledge, remind, escalate — the evidence is the point

• Defensibility is about **proof**, not just the hold: issue the notice, require **acknowledgment**, and **remind/escalate** non-responders on legal's cadence — that trail is what a court or regulator expects.

• Reuse a **standard set** of notice objects (`– Issuance` / `– Reminder` / `– Escalation` / `– Release`) with legal's text; the program **configures and tracks** them — legal **authors** them.

• Keep the **hold register** current as the single defensible record: who is on hold, since when, noticed, acknowledged, reminded, escalated, released. Export it on a cadence, not just at the end.

## Release is the one irreversible act — gate it, never script it

• **Releasing a hold deletes the preservation** and cannot be undone for content purged afterward. Release **only** on the named **release authority's** written sign-off (Req 2.5), **in the portal**, with the release recorded and the `– Release` notice sent.

• **No script in this pack releases a hold** — by design. If tooling ever *could* release, a mis-run is a spoliation event; keeping release out of automation is the control.

• Before releasing, confirm **no other matter** still requires the same location — overlapping holds mean one matter's release must not un-preserve content another still needs.

## Respect the legal-vs-technical boundary

• **Legal owns the determination; the program owns the execution and the evidence.** *Whether* a duty has

attached, *who* is a custodian, the *scope*, the notice's sufficiency, and *when* to release are eDiscovery-counsel calls — don't encode a legal judgement into a hold configuration.

• This program stops at **defensible preservation** — collection, review, analytics, and export are the

downstream eDiscovery phases; hand validated, preserved custodians to whatever review/production process legal runs.

### Licensing & portal reality

• **Custodian management and communications are Premium (E5-class)**; **case holds** work at **E3**. Verify

what your tenant is licensed for **before** designing a custodian program — and design a case-hold-only program on Standard rather than promising features the tenant can't run.

• The eDiscovery portal is **consolidating** into a new unified experience; tab names/paths (e.g. *Custodians*

→ *Data sources*) differ between classic and unified. **Verify the exact path in your tenant** — don't assume a UI that may have moved.

• **Commercial vs Government** clouds differ (portal URLs, `Connect-IPPSSession -ConnectionUri`, Graph

national-cloud endpoints, feature availability in GCC/GCC High/DoD) — verify the path and feature in your cloud.

### Operations & governance

• Give the program an **eDiscovery Administrator** owner and **least-privilege eDiscovery Managers** per

matter; **separate** the person who places holds from the person who authorizes release.

• Keep a **change log** of every placement, scope change, and release with the sign-off attached — the change

record and the register together are the defensible story.

• Re-run **Discovery/Validate/Report** on a cadence: confirm distribution stays Success, catch custodians

whose data sources changed, and keep the register current as matters age.

• Enable and retain **unified audit logging** — the tracking report and the evidence trail read from it; if

it's off, defensibility has no source.

## 8. Deliverables Summary — Legal-Hold & Custodian Management Program (Microsoft Purview)

What you receive in this solution, and what "done" looks like.

## Documentation

| Document | What it gives you |
|---|---|
| **Solution Overview** | The legal-hold + custodian program model — the hold-type distinction (case hold vs custodian-based hold vs Litigation Hold vs retention), how cases/holds/custodians/notices/tracking fit together, the deferred legal boundary and the MSEDISC (readiness) boundary, and the naming convention for every case/hold/custodian/notice object. |
| **Requirements Guide** | Solution-specific prerequisites — eDiscovery **Standard vs Premium** licensing, eDiscovery **Manager/Administrator** role groups, in-scope custodians/data-sources/non-custodial locations, the preservation scope + notice content from legal, a named **release authority** + change control, and audit logging + a pilot matter — each with verify/implement steps, on top of the six category prerequisites. |
| **Implementation Guide** | The exact ordered phases: connect/baseline → design the case + scope → configure the **case hold** (preview → commit) → add custodians + issue the notice (Premium) → validate distribution/coverage/acknowledgment → track and expand to real matters; each phase named, gated, with a rollback — and **no automated release**. |
| **Testing & Validation** | Pass/fail gates for the three headline outcomes — the hold **preserves** (A), custodians/notices are **covered and tracked** (B, Premium), and the program is **operable and defensible** (C) — plus overlapping-hold safety, pilot isolation, and no-auto-release checks, all on a pilot matter with **fabricated** custodians. |
| **Troubleshooting Guide** | The common failure modes (hold not distributing, custodian data source not mapping, notice not acknowledged, Premium features missing, tracking report empty) and how to resolve them. |
| **Best Practices** | Pilot-first discipline, preserve-first scoping, distribution validation, the notice/acknowledgment evidence trail, the release-is-irreversible rule, the legal-vs-technical boundary, and licensing/portal reality. |
| **Executive Summary** | One-page business framing for approvers. |

## PowerShell tools (5) — Security & Compliance PowerShell / Graph

All tools are **read-only or `-WhatIf` by default**; writes require an explicit `-Commit`. **No tool releases a hold** — release is a deliberate portal + sign-off action (Req 2.5). No credentials are stored in code. They connect via `Connect-IPPSSession` and/or `Connect-MgGraph` (eDiscovery, some endpoints **beta** — verify).

| # | Script | Role | Default / run-mode |
|---|---|---|---|
| 1 | **EDHOLD-Discovery.ps1** | eDiscovery posture assessment: existing cases, case hold policies and their **distribution status**, custodian/communications signal (Graph, where licensed), a **Premium-licensing** signal, and eDiscovery role-group membership. | **Read-only** |
| 2 | **EDHOLD-Config.ps1** | Hold/custodian config helper: creates/updates an eDiscovery **case hold policy + rule**, adds a **custodian** (Graph, where licensed), or enables Litigation Hold — `-WhatIf` **by default**; writes need `-Commit`. **Never releases a hold.** | `-WhatIf` (writes need `-Commit`) |
| 3 | **EDHOLD-Validate.ps1** | Validation checker: confirms hold **distribution = Success** across locations, expected custodians/locations in scope, Litigation Hold state, and (Graph, where exposed) notice/acknowledgment signal — per-gate pass/fail. | **Read-only** |

| # | Script | Role | Default / run-mode |
|---|---|---|---|
| 4 | **EDHOLD-Report.ps1** | Tracking report: exports the **hold/custodian register** — custodians, data sources, hold/distribution status, and notice/acknowledgment status/dates where exposed. | **Read-only** |
| 5 | **EDHOLD-Set-Scope.ps1** | Scope/health helper: reports a hold's locations/custodians and (with `-Commit`) **adds/adjusts** hold locations / custodian data sources. **Never releases a hold.** | `-WhatIf` (writes need `-Commit`) |

## Definition of done

• eDiscovery licensing tier confirmed (Standard = case holds; Premium = custodians + communications) and

eDiscovery role-group ownership assigned (Administrator + per-matter Managers).

• A **pilot matter** case with a **case hold policy + rule** built (preview → commit), scoped to **fabricated test** locations, at the preservation scope legal chose (everything vs a validated query).

• **Custodians** (Premium) added with mapped data sources, **non-custodial** locations held, and the **hold notice** issued with acknowledgment tracked and reminder/escalation configured.

• Hold **distribution = Success** validated across all pilot locations, deleted held content proven recoverable, and every intended custodian/location confirmed in scope (Gate A/B).

• The **hold register** exported and current; the evidence/audit trail present; role separation and a named **release authority** in place (Gate C).

• Pilot isolation, safe teardown, and **no-auto-release** verified (S1–S4); the program minted to production

naming for real matters under change control.

> ***Boundary:*** *the solution delivers the technical program and the evidence.* ***Whether a duty has attached, who is a custodian, the preservation scope, a notice's legal sufficiency, and when a hold may be released*** *are legal / eDiscovery-counsel determinations, captured alongside the evidence — and* ***no delivered tool releases a hold****.*

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## EDHOLD-Config.ps1

**Purpose:** One safe-by-default helper for the write actions of the hold program: -Action NewCaseHold : New-CaseHoldPolicy on an eDiscovery case, scoped to the given locations. -Action NewCaseHoldRule : New-CaseHoldRule on a policy (empty ContentMatchQuery = preserve-everything; a KQL query = query-based, as authored by legal). -Action EnableLitigationHold: Set-Mailbox -LitigationHoldEnabled $true (belt-and-suspenders mailbox hold). -Action AddCustodian : add a custodian via the Microsoft Graph eDiscovery API (Premium; beta — if unavailable in your tenant, add the custodian in the Purview portal). -Action RemoveCaseHold : Remove a PILOT case hold policy/rule (test custodians only). Safe by default: with -WhatIf (the default when -Commit is absent) it only reports the intended change; it applies nothing unless -Commit is passed. It NEVER releases a hold on a real matter — hold release is a deliberate portal + release-authority sign-off action (Req 2.5). Use RemoveCaseHold only to tear down a PILOT hold on fabricated custodians. HOW TO RUN Preview a pilot case hold policy: powershell -File .\EDHOLD-Config.ps1 -Action NewCaseHold `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' `-ExchangeLocation 'testcustodian1@contoso.com' -WhatIf Apply it (add -Commit). Then create a preserve-everything rule: powershell -File .\EDHOLD-Config.ps1 -Action NewCaseHoldRule` -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' ` -HoldRuleName 'Hold Rule – Contoso-Matter-001 – Pilot' -ContentMatchQuery '' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession); AddCustodian also needs Graph. .EXAMPLE .\EDHOLD-Config.ps1 -Action NewCaseHold -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -ExchangeLocation 'testcustodian1@contoso.com' -WhatIf Prints the New-CaseHoldPolicy call it WOULD run. Re-run with -Commit to apply. .NOTES Purview — Legal-Hold & Custodian Management Program. Writes gated behind -Commit; NEVER releases a real hold. Expected results: -Commit creates/updates the pilot hold; verify DistributionStatus=Success with EDHOLD-Validate.ps1 (PASS). > [CmdletBinding()] param( [Parameter(Mandatory)] [ValidateSet('NewCaseHold','NewCaseHoldRule','EnableLitigationHold','AddCustodian','RemoveCaseHold')] [string]$Action,

**How to run:** Preview a pilot case hold policy: powershell -File .\EDHOLD-Config.ps1 -Action NewCaseHold `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' `-ExchangeLocation 'testcustodian1@contoso.com' -WhatIf Apply it (add -Commit). Then create a preserve-everything rule: powershell -File .\EDHOLD-Config.ps1 -Action NewCaseHoldRule` -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' ` -HoldRuleName 'Hold Rule – Contoso-Matter-001 – Pilot' -ContentMatchQuery '' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession); AddCustodian also needs Graph. .EXAMPLE .\EDHOLD-Config.ps1 -Action NewCaseHold -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -ExchangeLocation 'testcustodian1@contoso.com' -WhatIf Prints the New-CaseHoldPolicy call it WOULD run. Re-run with -Commit to apply. .NOTES Purview — Legal-Hold & Custodian Management Program. Writes gated behind -Commit; NEVER releases a real hold. Expected results: -Commit creates/updates the pilot hold; verify DistributionStatus=Success with EDHOLD-Validate.ps1 (PASS). > [CmdletBinding()] param( [Parameter(Mandatory)] [ValidateSet('NewCaseHold','NewCaseHoldRule','EnableLitigationHold','AddCustodian','RemoveCaseHold')] [string]$Action,

## EDHOLD-Discovery.ps1

**Purpose:** Builds the legal-hold baseline before you configure anything: every eDiscovery ComplianceCase, every CaseHoldPolicy with its held locations and DISTRIBUTION STATUS (Pending vs Success vs error), the CaseHoldRules and whether they are query-based or preserve-everything, plus eDiscovery role-group membership and a Premium-licensing signal (whether custodian/communications features are likely available). Optionally probes the Microsoft Graph eDiscovery API for custodians where licensed (beta — verify in your tenant). Read-only — makes no changes and NEVER releases a hold. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EDHOLD-Discovery.ps1 -CsvPath .\EDHOLD-Posture.csv Requires Security & Compliance PowerShell (Connect-IPPSSession via ExchangeOnlineManagement); optional Microsoft Graph (Connect-MgGraph) for custodian read. .EXAMPLE .\EDHOLD-Discovery.ps1 -CsvPath .\EDHOLD-Posture.csv Prints cases, case hold policies (locations + distribution status), rules, and role membership; exports CSV. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only against Security & Compliance PowerShell. Expected results: holds you expect report DistributionStatus=Success across their locations (PASS); anything Pending or errored is flagged to validate before you trust it. > [CmdletBinding()] param( [string]$CsvPath, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )

**How to run:** powershell -ExecutionPolicy Bypass -File .\EDHOLD-Discovery.ps1 -CsvPath .\EDHOLD-Posture.csv Requires Security & Compliance PowerShell (Connect-IPPSSession via ExchangeOnlineManagement); optional Microsoft Graph (Connect-MgGraph) for custodian read. .EXAMPLE .\EDHOLD-Discovery.ps1 -CsvPath .\EDHOLD-Posture.csv Prints cases, case hold policies (locations + distribution status), rules, and role membership; exports CSV. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only against Security & Compliance PowerShell. Expected results: holds you expect report DistributionStatus=Success across their locations (PASS); anything Pending or errored is flagged to validate before you trust it. > [CmdletBinding()] param( [string]$CsvPath, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )

## EDHOLD-Report.ps1

**Purpose:** Produces the defensible "Custodian Hold Register" for a matter: for the case's CaseHoldPolicies, the held locations, distribution status, and rule scope (query vs preserve-everything); and — where eDiscovery Premium is licensed and the Microsoft Graph eDiscovery API is available (beta) — each custodian, their mapped data sources, hold status, and (where the API exposes it) notice/acknowledgment status. Read-only — makes no changes. Notice ISSUANCE/ACKNOWLEDGMENT is operated in the Purview portal (Communications); this report inventories status, it does not send or acknowledge notices. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EDHOLD-Report.ps1 `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -CsvPath '.\Custodian Hold Register – Contoso-Matter-001 – Report.csv' Requires Security & Compliance PowerShell (Connect-IPPSSession); custodian rows require Graph (Premium/beta). .EXAMPLE .\EDHOLD-Report.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -CsvPath '.\register.csv' Prints the hold/custodian register and exports it as the tracking record for the matter. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only tracking report. No changes made. Expected results: the register matches the portal state (holds, locations, custodians, notice/ack) — keep it current as the defensible record of the matter (PASS = register reflects reality). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$CaseName, [string]$CsvPath, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )

**How to run:** powershell -ExecutionPolicy Bypass -File .\EDHOLD-Report.ps1 `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -CsvPath '.\Custodian Hold Register – Contoso-Matter-001 – Report.csv' Requires Security & Compliance PowerShell (Connect-IPPSSession); custodian rows require Graph (Premium/beta). .EXAMPLE .\EDHOLD-Report.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -CsvPath '.\register.csv' Prints the hold/custodian register and exports it as the tracking record for the matter. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only tracking report. No changes made. Expected results: the register matches the portal state (holds, locations, custodians, notice/ack) — keep it current as the defensible record of the matter (PASS = register reflects reality). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$CaseName, [string]$CsvPath, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )

## EDHOLD-Set-Scope.ps1

**Purpose:** Reports a case hold policy's current locations, distribution status, and rule scope, and — only with -Commit — ADDS Exchange / OneDrive / SharePoint locations to the hold (Set-CaseHoldPolicy -Add*), so a hold can be widened as a custodian's role or data sources change. Safe by default: -WhatIf reports only; writes require -Commit. This helper only ADDS/adjusts coverage — it does NOT remove a location and NEVER releases a hold. Removing a location or releasing a hold reduces preservation and is a deliberate portal + release-authority sign-off action (Req 2.5); it is intentionally not automated here. HOW TO RUN Report current scope/health: powershell -File .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -WhatIf Add a newly-relevant test custodian mailbox to the hold (widen coverage — still same hold): powershell -File .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' ` -AddExchangeLocation 'testcustodian2@contoso.com' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -WhatIf Prints the hold's locations/status and the Set-CaseHoldPolicy -Add* call it WOULD run. .NOTES Purview — Legal-Hold & Custodian Management Program. Additive scope changes gated behind -Commit; never removes a location, never releases a hold. Expected results: -Commit ADDS the location; re-validate DistributionStatus=Success with EDHOLD-Validate.ps1 (PASS = wider hold, still distributing to Success). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$HoldPolicyName, [string[]]$AddExchangeLocation, [string[]]$AddOneDriveLocation, [string[]]$AddSharePointLocation, [string]$UserPrincipalName, [switch]$Commit )

**How to run:** Report current scope/health: powershell -File .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -WhatIf Add a newly-relevant test custodian mailbox to the hold (widen coverage — still same hold): powershell -File .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' ` -AddExchangeLocation 'testcustodian2@contoso.com' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\EDHOLD-Set-Scope.ps1 -HoldPolicyName 'Hold Policy – Contoso-Matter-001 – Pilot' -WhatIf Prints the hold's locations/status and the Set-CaseHoldPolicy -Add* call it WOULD run. .NOTES Purview — Legal-Hold & Custodian Management Program. Additive scope changes gated behind -Commit; never removes a location, never releases a hold. Expected results: -Commit ADDS the location; re-validate DistributionStatus=Success with EDHOLD-Validate.ps1 (PASS = wider hold, still distributing to Success). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$HoldPolicyName, [string[]]$AddExchangeLocation, [string[]]$AddOneDriveLocation, [string[]]$AddSharePointLocation, [string]$UserPrincipalName, [switch]$Commit )

## EDHOLD-Validate.ps1

**Purpose:** Confirms, against explicit per-gate PASS/FAIL, that the hold for a matter is actually effective: Gate A — every CaseHoldPolicy on the case reports DistributionStatus = Success (the hold has propagated to all its locations; Pending means "not yet — wait, don't trust"). Gate B — the expected custodian mailboxes / OneDrive / SharePoint locations are actually in the hold's location list (nothing intended is missing). Gate C — where -ExpectedExchangeLocation mailboxes are given, their Litigation Hold state is reported (belt-and-suspenders reinforcement, where used). Gate D — (optional) Graph eDiscovery custodian/hold signal where Premium is licensed (beta). Read-only — queries only; makes NO changes and never releases a hold. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EDHOLD-Validate.ps1 `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -ExpectedExchangeLocation 'testcustodian1@contoso.com' Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\EDHOLD-Validate.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -ExpectedExchangeLocation 'testcustodian1@contoso.com' Prints PASS/FAIL for distribution status, expected-location coverage, and Litigation Hold state. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only validation. Expected results: PASS = DistributionStatus=Success on every policy AND every expected location is held. A Pending distribution is not a failure yet — re-run after the service timers complete (minutes to hours). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$CaseName, [string[]]$ExpectedExchangeLocation, [string[]]$ExpectedSharePointLocation, [string[]]$ExpectedOneDriveLocation, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )

**How to run:** powershell -ExecutionPolicy Bypass -File .\EDHOLD-Validate.ps1 `-CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot'` -ExpectedExchangeLocation 'testcustodian1@contoso.com' Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\EDHOLD-Validate.ps1 -CaseName 'Legal Hold Case – Contoso-Matter-001 – Pilot' -ExpectedExchangeLocation 'testcustodian1@contoso.com' Prints PASS/FAIL for distribution status, expected-location coverage, and Litigation Hold state. .NOTES Purview — Legal-Hold & Custodian Management Program. Read-only validation. Expected results: PASS = DistributionStatus=Success on every policy AND every expected location is held. A Pending distribution is not a failure yet — re-run after the service timers complete (minutes to hours). > [CmdletBinding()] param( [Parameter(Mandatory)][string]$CaseName, [string[]]$ExpectedExchangeLocation, [string[]]$ExpectedSharePointLocation, [string[]]$ExpectedOneDriveLocation, [string]$UserPrincipalName, [switch]$IncludeGraphCustodians )